



重慶大學
CHONGQING UNIVERSITY

操作及部署文档

项目名称：基于多智能体协作的统计图表到代码自动生成研究与实现

项目编号：202610611118

所属学院：大数据与软件学院

项目负责人：周怀涛

联系电话：18325095023

项目组成员：周怀涛，汤英琦，谢家旭

指导教师：徐玲

多智能体图表复现系统 - 部署与操作手册

版本: v3.2.3

更新日期: 2026年4月

适用平台: Windows / Linux / macOS

目录

- 系统概述
- 环境要求
- 环境配置
- 依赖安装
- 系统配置
- 服务启动
- 功能使用
- 运行流程
- 常见问题排查
- 性能优化
- 安全建议
- 维护与监控

1. 系统概述

1.1 项目简介

多智能体图表复现系统是一个基于多模态大模型的智能图表复现平台，通过多智能体协作自动生成高质量的 Matplotlib 代码。系统采用前后端分离架构，提供 Web 可视化界面和命令行工具两种使用方式。

1.2 核心功能

- 多智能体协作流水线:** 代码生成 → 视觉评估 → 代码分析 → 反馈修订
- 多维度验证系统:** 颜色、文本、结构一致性和视觉感知综合评分
- 多模型支持:** 通义千问 (Qwen)、OpenAI GPT、Google Gemini、字节豆包 (Doubao)
- 实时进度监控:** WebSocket 实时推送流水线执行状态
- 结果对比分析:** 可视化展示原图与生成图的对比效果

1.3 技术架构

后端技术栈:

- FastAPI: 高性能异步 Web 框架
- Matplotlib: 图表渲染引擎
- Pytesseract: OCR 文字识别
- WebSocket: 实时通信

前端技术栈:

- Vue 3: 渐进式 JavaScript 框架
- Vite: 新一代前端构建工具
- Pinia: Vue 状态管理库
- Chart.js / ECharts: 数据可视化

AI 模型:

- 支持多家主流大模型服务商
- 统一的模型调用接口
- 灵活的模型切换机制

2. 环境要求

2.1 硬件要求

配置项	最低要求	推荐配置
CPU	双核 2.0GHz	四核 3.0GHz+
内存	4GB	8GB+
硬盘	5GB 可用空间	20GB+ SSD
网络	稳定的互联网连接	带宽 10Mbps+

2.2 软件要求

软件	版本要求	说明
Python	3.9 - 3.11	核心运行环境
Node.js	16.x - 20.x	前端开发环境
npm	8.x+	Node 包管理器
Tesseract OCR	4.x+	文字识别引擎
Git	2.x+	版本控制（可选）

2.3 操作系统支持

-  Windows 10/11 (x64)
-  Ubuntu 20.04+ / Debian 11+
-  macOS 11+ (Big Sur 及以上)
-  CentOS 8+ / RHEL 8+

2.4 浏览器要求

推荐使用以下现代浏览器访问 Web 界面：

- Chrome 90+
- Firefox 88+

- Edge 90+
- Safari 14+

3. 环境配置

3.1 Python 环境配置

3.1.1 检查 Python 版本

```
# 检查 Python 版本
python --version
# 或
python3 --version

# 应输出: Python 3.9.x 或更高版本
```

3.1.2 安装 Python (如未安装)

Windows:

1. 访问 [Python 官网](#)
2. 下载 Python 3.9+ 安装包
3. 运行安装程序, 勾选 "Add Python to PATH"
4. 选择 "Install Now" 完成安装

Linux (Ubuntu/Debian):

```
sudo apt update
sudo apt install python3.9 python3.9-venv python3-pip
```

macOS:

```
# 使用 Homebrew
brew install python@3.9
```

3.1.3 创建虚拟环境 (推荐)

使用虚拟环境可以隔离项目依赖, 避免版本冲突:

```
# 进入项目目录
cd MultiAgentFrame-main

# 创建虚拟环境
```

```
python -m venv venv

# 激活虚拟环境
# Windows
venv\Scripts\activate

# Linux/macOS
source venv/bin/activate

# 激活后，命令行前会显示 (venv) 标识
```

3.2 Node.js 环境配置

3.2.1 检查 Node.js 版本

```
# 检查 Node.js 版本
node --version
# 应输出: v16.x.x 或更高

# 检查 npm 版本
npm --version
# 应输出: 8.x.x 或更高
```

3.2.2 安装 Node.js (如未安装)

Windows / macOS:

1. 访问 [Node.js 官网](#)
2. 下载 LTS 版本安装包
3. 运行安装程序，按默认选项安装

Linux (Ubuntu/Debian):

```
# 使用 NodeSource 仓库安装 Node.js 18.x
curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -
sudo apt-get install -y nodejs
```

使用 nvm 管理多版本 (推荐) :

```
# 安装 nvm
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.0/install.sh | bash

# 安装 Node.js 18
nvm install 18
nvm use 18
```

3.3 Tesseract OCR 配置

Tesseract 是系统进行文字识别的核心组件，必须正确安装。

3.3.1 Windows 安装

1. 访问 [Tesseract GitHub Releases](#)
2. 下载最新的 Windows 安装包（如 `tesseract-ocr-w64-setup-5.3.x.exe`）
3. 运行安装程序，记住安装路径（默认: `C:\Program Files\Tesseract-OCR`）
4. 安装完成后，记录 `tesseract.exe` 的完整路径

验证安装:

```
# 将 Tesseract 添加到 PATH（临时）
set PATH=%PATH%;C:\Program Files\Tesseract-OCR

# 验证安装
tesseract --version
```

3.3.2 Linux 安装

Ubuntu/Debian:

```
sudo apt update
sudo apt install tesseract-ocr
sudo apt install libtesseract-dev

# 安装中文语言包（可选）
sudo apt install tesseract-ocr-chi-sim
```

CentOS/RHEL:

```
sudo yum install epel-release
sudo yum install tesseract
```

验证安装:

```
tesseract --version
which tesseract # 查看安装路径
```

3.3.3 macOS 安装

```
# 使用 Homebrew 安装
brew install tesseract

# 验证安装
tesseract --version
which tesseract # 通常为 /usr/local/bin/tesseract
```

4. 依赖安装

4.1 获取项目代码

4.1.1 从 Git 仓库克隆

```
# 克隆项目
git clone <repository-url>
cd MultiAgentFrame-main
```

4.1.2 从压缩包解压

```
# 解压项目文件
unzip MultiAgentFrame-main.zip
cd MultiAgentFrame-main
```

4.2 后端依赖安装

4.2.1 安装核心依赖

确保已激活虚拟环境，然后安装依赖：

```
# 确认在项目根目录
pwd # Linux/macOS
cd # Windows

# 安装后端依赖
pip install -r backend/requirements.txt

# 如果速度较慢，可使用国内镜像
pip install -r backend/requirements.txt -i
https://pypi.tuna.tsinghua.edu.cn/simple
```

4.2.2 验证安装

```
# 验证关键包是否安装成功
python -c "import fastapi; print('FastAPI:', fastapi.__version__)"
python -c "import uvicorn; print('Uvicorn:', uvicorn.__version__)"
python -c "import pytesseract; print('Pytesseract: OK')"
python -c "import matplotlib; print('Matplotlib:', matplotlib.__version__)"
```

4.2.3 依赖说明

包名	版本	用途
fastapi	0.104.1	Web 框架
uvicorn	0.24.0	ASGI 服务器
python-multipart	0.0.6	文件上传支持
python-dotenv	1.0.0	环境变量管理
pydantic	2.5.0	数据验证
websockets	12.0	WebSocket 支持

4.3 前端依赖安装

4.3.1 安装 npm 依赖

```
# 进入前端目录
cd frontend

# 安装依赖
npm install

# 如果速度较慢, 可使用国内镜像
npm install --registry=https://registry.npmmirror.com

# 返回项目根目录
cd ..
```

4.3.2 验证安装

```
# 检查 node_modules 是否生成
ls frontend/node_modules # Linux/macOS
dir frontend\node_modules # Windows

# 查看已安装的包
cd frontend
npm list --depth=0
```


4.3.3 依赖说明

包名	版本	用途
vue	^3.5.32	前端框架
vue-router	^4.6.4	路由管理
pinia	^3.0.4	状态管理
chart.js	^4.5.1	图表库
echarts	^6.0.0	数据可视化
vite	^8.0.10	构建工具

5. 系统配置

5.1 环境变量配置

5.1.1 创建配置文件

在项目根目录创建 `.env` 文件：

```
# Windows
copy .env.example .env

# Linux/macOS
cp .env.example .env
```

如果没有 `.env.example`，手动创建 `.env` 文件。

5.1.2 配置模板

```
# ===== 模型配置 =====
# 当前使用的模型提供商 (qwen, openai, gemini, doubao)
MODEL_PROVIDER=qwen

# ===== API Keys =====
# 阿里云通义千问 (Qwen)
QWEN_API_KEY=your_qwen_api_key_here
DASHSCOPE_API_KEY=your_qwen_api_key_here # 兼容旧配置

# OpenAI GPT
# OPENAI_API_KEY=your_openai_api_key_here
# OPENAI_BASE_URL=https://api.openai.com/v1

# Google Gemini
# GEMINI_API_KEY=your_gemini_api_key_here
```

```
# 字节跳动豆包 (Doubao)
# DOUBAO_API_KEY=your_doubao_api_key_here
# DOUBAO_BASE_URL=https://ark.cn-beijing.volces.com/api/v3

# ===== 服务器配置 =====
SERVER_HOST=localhost
SERVER_PORT=8001

# ===== Tesseract 配置 =====
# Windows 示例
TESSERACT_CMD=C:\Program Files\Tesseract-OCR\tesseract.exe
# Linux/macOS 示例
# TESSERACT_CMD=/usr/bin/tesseract
```

5.1.3 获取 API Key

通义千问 (Qwen):

1. 访问 [阿里云 DashScope](#)
2. 登录/注册阿里云账号
3. 进入控制台 → API-KEY 管理
4. 创建新的 API Key 并复制

OpenAI GPT:

1. 访问 [OpenAI Platform](#)
2. 登录/注册账号
3. 进入 API Keys 页面
4. 创建新的 API Key 并复制

Google Gemini:

1. 访问 [Google AI Studio](#)
2. 登录 Google 账号
3. 获取 API Key

字节豆包 (Doubao):

1. 访问 [火山引擎](#)
2. 登录/注册账号
3. 进入豆包大模型服务
4. 创建 API Key

5.2 前端配置

5.2.1 前端环境变量

编辑 `frontend/.env` 文件:

```
# API 配置
VITE_API_BASE_URL=http://localhost:8001/api
VITE_WS_URL=ws://localhost:8001/ws

# 上传配置
VITE_MAX_FILE_SIZE=10485760 # 10MB
```

5.2.2 配置说明

- **VITE_API_BASE_URL**: 后端 API 地址
- **VITE_WS_URL**: WebSocket 连接地址
- **VITE_MAX_FILE_SIZE**: 最大上传文件大小（字节）

5.3 目录结构初始化

系统运行时会自动创建必要的目录，也可以手动创建：

```
# 创建存储目录
mkdir -p storage/uploads
mkdir -p storage/outputs
mkdir -p backend/logs

# Windows
mkdir storage\uploads
mkdir storage\outputs
mkdir backend\logs
```

6. 服务启动

6.1 一键启动（推荐）

使用提供的启动脚本同时启动前后端服务：

```
# 确保在项目根目录
python scripts/start_service.py
```

启动成功后会显示：

```
=====
🚀 多智能体图表复现系统 - 全栈服务启动
=====

🚀 后端服务启动中...
📁 项目根目录: /path/to/MultiAgentFrame-main
🌐 API 文档: http://localhost:8001/docs
```

```
🔗 健康检查: http://localhost:8001/health
=====
🎨 前端服务启动中...
📁 前端目录: /path/to/MultiAgentFrame-main/frontend
🌐 前端地址: http://localhost:5174
=====
```

6.2 分别启动

如果需要分别启动前后端服务（用于开发调试）：

6.2.1 启动后端服务

方式一：使用 `uvicorn` 命令

```
# 进入后端目录
cd backend

# 启动服务（开发模式，支持热重载）
uvicorn main:app --reload --host 0.0.0.0 --port 8001

# 生产模式（不支持热重载，性能更好）
uvicorn main:app --host 0.0.0.0 --port 8001 --workers 4
```

方式二：直接运行 `main.py`

```
# 在项目根目录
python backend/main.py
```

验证后端启动:

```
# 健康检查
curl http://localhost:8001/health

# 应返回: {"status":"ok"}
```

6.2.2 启动前端服务

打开新的终端窗口：

```
# 进入前端目录
cd frontend

# 启动开发服务器
```

```
npm run dev

# 前端会在 http://localhost:5174 启动
```

验证前端启动:

- 浏览器访问 <http://localhost:5174>
- 应该能看到系统首页

6.3 生产环境部署

6.3.1 构建前端

```
cd frontend

# 构建生产版本
npm run build

# 构建产物在 frontend/dist 目录
```

6.3.2 使用 Nginx 部署

安装 Nginx:

```
# Ubuntu/Debian
sudo apt install nginx

# CentOS/RHEL
sudo yum install nginx
```

配置 Nginx (/etc/nginx/sites-available/multiagent):

```
server {
    listen 80;
    server_name your-domain.com;

    # 前端静态文件
    location / {
        root /path/to/MultiAgentFrame-main/frontend/dist;
        try_files $uri $uri/ /index.html;
    }

    # 后端 API 代理
    location /api {
        proxy_pass http://localhost:8001;
        proxy_http_version 1.1;
    }
}
```

```
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection 'upgrade';
    proxy_set_header Host $host;
    proxy_cache_bypass $http_upgrade;
}

# WebSocket 代理
location /ws {
    proxy_pass http://localhost:8001;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "Upgrade";
    proxy_set_header Host $host;
}
}
```

启用配置:

```
sudo ln -s /etc/nginx/sites-available/multiagent /etc/nginx/sites-enabled/
sudo nginx -t # 测试配置
sudo systemctl restart nginx
```

6.3.3 使用 systemd 管理后端服务

创建服务文件 `/etc/systemd/system/multiagent-backend.service`:

```
[Unit]
Description=MultiAgent Backend Service
After=network.target

[Service]
Type=simple
User=www-data
WorkingDirectory=/path/to/MultiAgentFrame-main
Environment="PATH=/path/to/venv/bin"
ExecStart=/path/to/venv/bin/uvicorn backend.main:app --host 0.0.0.0 --port 8001 --workers 4
Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target
```

启动服务:

```
sudo systemctl daemon-reload
sudo systemctl enable multiagent-backend
```

```
sudo systemctl start multiagent-backend
sudo systemctl status multiagent-backend
```

6.4 Docker 部署（可选）

6.4.1 创建 Dockerfile

在项目根目录创建 **Dockerfile**:

```
FROM python:3.9-slim

# 安装系统依赖
RUN apt-get update && apt-get install -y \
    tesseract-ocr \
    libtesseract-dev \
    && rm -rf /var/lib/apt/lists/*

# 设置工作目录
WORKDIR /app

# 复制依赖文件
COPY backend/requirements.txt .

# 安装 Python 依赖
RUN pip install --no-cache-dir -r requirements.txt

# 复制项目文件
COPY . .

# 暴露端口
EXPOSE 8001

# 启动命令
CMD ["uvicorn", "backend.main:app", "--host", "0.0.0.0", "--port", "8001"]
```

6.4.2 创建 docker-compose.yml

```
version: '3.8'

services:
  backend:
    build: .
    ports:
      - "8001:8001"
    environment:
      - MODEL_PROVIDER=qwen
      - QWEN_API_KEY=${QWEN_API_KEY}
      - TESSERACT_CMD=/usr/bin/tesseract
    volumes:
```

```
- ./storage:/app/storage
- ./backend/logs:/app/backend/logs
restart: unless-stopped

frontend:
  image: node:18-alpine
  working_dir: /app
  volumes:
    - ./frontend:/app
  ports:
    - "5174:5174"
  command: npm run dev
  depends_on:
    - backend
```

启动容器:

```
docker-compose up -d
```

7. 功能使用

7.1 Web 界面使用

7.1.1 访问系统

- 1. 确保前后端服务已启动
- 2. 打开浏览器访问: <http://localhost:5174>
- 3. 系统首页会显示欢迎界面

7.1.2 上传图表

- 1. 点击 "上传图表" 或 "开始使用" 按钮
- 2. 选择要复现的图表图片文件
 - 支持格式: PNG, JPG, JPEG
 - 文件大小: 最大 10MB
 - 建议分辨率: 800x600 以上
- 3. 图片上传成功后会显示预览

7.1.3 配置参数

在流水线配置页面设置以下参数：

参数	说明	默认值	推荐范围
最大迭代轮数	流水线最多执行的轮数	5	3-10
验证阈值	通过验证的最低分数	0.75	0.6-0.9

参数说明:

- **最大迭代轮数:** 越大生成质量越高，但耗时越长
- **验证阈值:** 越高要求越严格，可能需要更多轮次

7.1.4 启动流水线

1. 点击 "启动流水线" 按钮
2. 系统开始执行多智能体协作流程
3. 实时显示执行进度：
 - Agent 1: 代码生成
 - Agent 2: 视觉评估
 - Agent 3: 代码分析
 - Agent 4: 反馈修订
 - 多维验证器: 综合评分

7.1.5 查看结果

流水线完成后:

1. **对比视图:** 左侧原图，右侧生成图
2. **评分详情:**
 - 颜色一致性分数
 - 文本一致性分数
 - 结构一致性分数
 - VLM 感知分数
 - 综合得分
3. **代码查看:** 查看生成的 Matplotlib 代码
4. **下载结果:**
 - 下载生成的图片
 - 下载 Python 代码
 - 下载完整报告

7.1.6 历史记录

1. 点击 "历史记录" 菜单
2. 查看所有执行过的流水线
3. 可以重新查看任意历史结果
4. 支持删除不需要的记录

7.2 命令行工具使用

命令行工具适合批量处理和自动化场景。

7.2.1 基本用法

```
python scripts/cli.py -i <input_image> -o <output_dir>
```

7.2.2 完整参数

```
python scripts/cli.py \  
  --input input.png \  
  --out outputs \  
  --max-loops 5 \  
  --threshold 0.75 \  
  --model qwen
```

参数说明:

- **-i, --input**: 输入参考图路径（必需）
- **-o, --out**: 输出目录（默认: outputs）
- **--max-loops**: 最大迭代轮数（默认: 5）
- **--threshold**: 验证阈值（默认: 0.75）
- **--model**: 模型提供商（默认: 使用 .env 配置）

7.2.3 批量处理示例

```
# 处理单个文件  
python scripts/cli.py -i chart1.png -o results/chart1  
  
# 批量处理（使用 shell 脚本）  
for file in images/*.png; do  
  python scripts/cli.py -i "$file" -o "results/${basename $file .png}"  
done  
  
# Windows 批处理  
for %%f in (images\*.png) do (  
  python scripts\cli.py -i "%%f" -o "results\%%~nf"  
)
```

7.2.4 输出文件说明

执行完成后，输出目录包含以下文件：

```
outputs/  
├─ current_matplotlib.py      # 生成的 Matplotlib 代码  
├─ generated_chart.png        # 渲染的图表  
├─ report_agent2_round1.txt    # Agent2 视觉评估报告  
├─ report_agent3_round1.txt    # Agent3 代码分析报告  
├─ validator_round1.json       # 第1轮验证结果  
├─ validator_round2.json       # 第2轮验证结果（如有）  
└─ ...
```

7.3 API 接口使用

系统提供 RESTful API，可集成到其他应用。

7.3.1 API 文档

访问 <http://localhost:8001/docs> 查看完整的 API 文档（Swagger UI）。

7.3.2 主要接口

1. 上传图片

```
POST /api/upload
Content-Type: multipart/form-data

curl -X POST "http://localhost:8001/api/upload" \
-F "file=@chart.png"

# 响应
{
  "file_id": "abc123",
  "filename": "chart.png",
  "url": "http://localhost:8001/uploads/abc123.png"
}
```

2. 启动流水线

```
POST /api/pipeline/start
Content-Type: application/json

curl -X POST "http://localhost:8001/api/pipeline/start" \
-H "Content-Type: application/json" \
-d '{
  "file_id": "abc123",
  "max_loops": 5,
  "threshold": 0.75
}'

# 响应
{
  "pipeline_id": "pipeline_xyz789",
  "status": "running"
}
```

3. 查询流水线状态

```
GET /api/pipeline/{pipeline_id}/status
```

```
curl "http://localhost:8001/api/pipeline/pipeline_xyz789/status"

# 响应
{
  "pipeline_id": "pipeline_xyz789",
  "status": "completed",
  "current_round": 3,
  "max_loops": 5,
  "final_score": 0.82
}
```

4. 获取结果

```
GET /api/results/{pipeline_id}

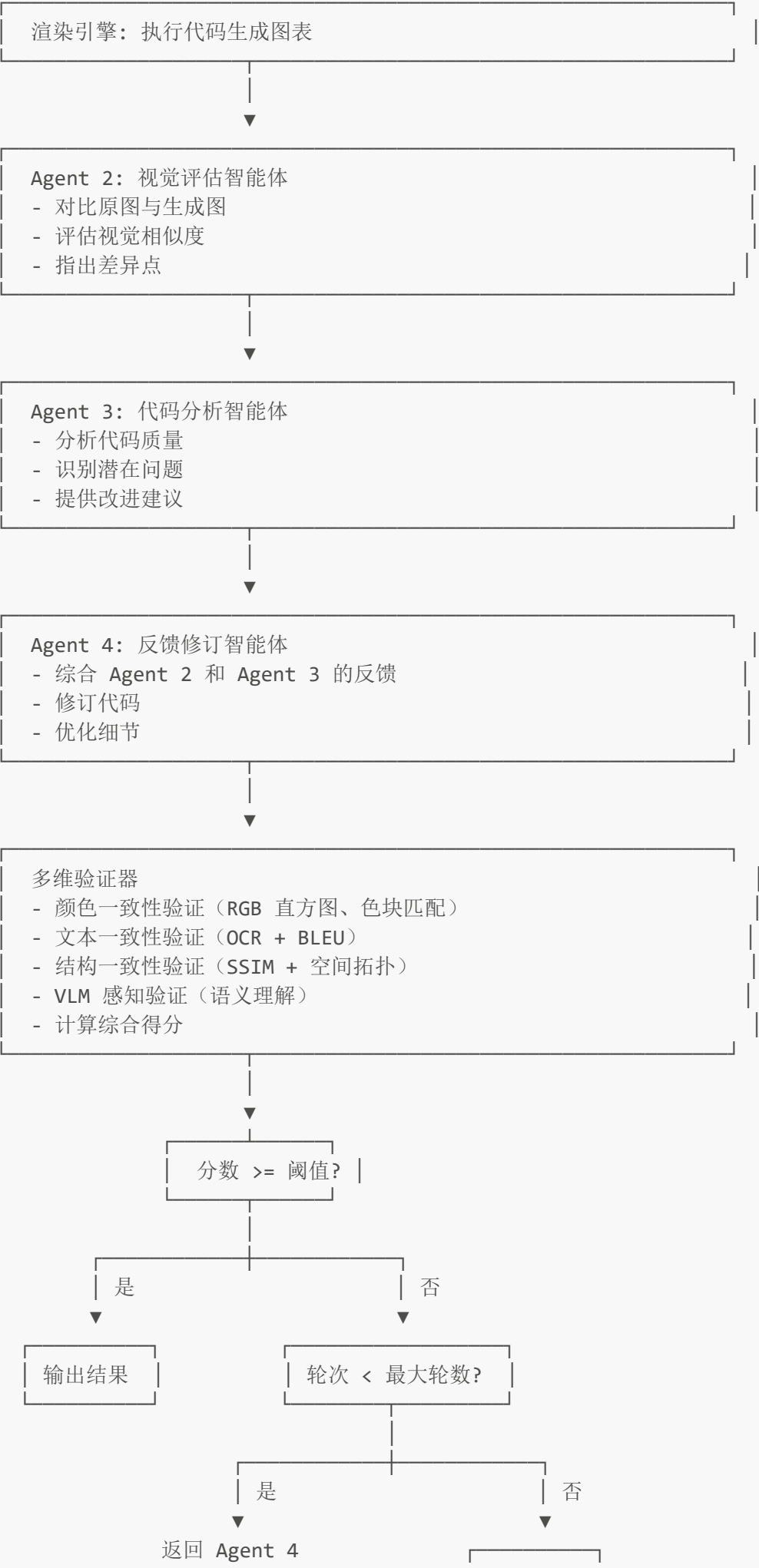
curl "http://localhost:8001/api/results/pipeline_xyz789"

# 响应
{
  "pipeline_id": "pipeline_xyz789",
  "reference_image": "http://localhost:8001/uploads/abc123.png",
  "generated_image": "http://localhost:8001/outputs/xyz789_generated.png",
  "code": "import matplotlib.pyplot as plt\n...",
  "scores": {
    "color": 0.85,
    "text": 0.78,
    "structure": 0.83,
    "vlm": 0.82,
    "overall": 0.82
  }
}
```

8. 运行流程

8.1 系统工作流程





进行下一轮迭代

输出结果

8.2 多维验证详解

8.2.1 颜色一致性验证

验证方法:

- RGB 直方图相似度计算
- 主色调提取与匹配
- 色块分布对比

评分标准:

- 0.9-1.0: 颜色几乎完全一致
- 0.7-0.9: 颜色基本一致，有细微差异
- 0.5-0.7: 颜色有明显差异
- 0.0-0.5: 颜色差异很大

8.2.2 文本一致性验证

验证方法:

- Tesseract OCR 提取文本
- BLEU 算法计算文本相似度
- 文本位置对比

评分标准:

- 0.9-1.0: 文本内容和位置完全一致
- 0.7-0.9: 文本内容基本一致
- 0.5-0.7: 部分文本缺失或错误
- 0.0-0.5: 文本差异很大

8.2.3 结构一致性验证

验证方法:

- SSIM (结构相似性指数) 计算
- 边缘检测与对比
- 空间拓扑分析

评分标准:

- 0.9-1.0: 结构完全一致
- 0.7-0.9: 结构基本一致
- 0.5-0.7: 结构有明显差异
- 0.0-0.5: 结构完全不同

8.2.4 VLM 感知验证

验证方法:

- 使用视觉语言模型进行语义理解
- 评估整体观感和细节
- 综合人类视觉感知

评分标准:

- 0.9-1.0: 视觉效果几乎完美
- 0.7-0.9: 视觉效果良好
- 0.5-0.7: 视觉效果一般
- 0.0-0.5: 视觉效果较差

8.2.5 综合评分

综合得分 = (颜色分数 × 0.25) + (文本分数 × 0.25) + (结构分数 × 0.25) + (VLM分数 × 0.25)

8.3 执行时间估算

场景	图表复杂度	迭代轮数	预计耗时
简单图表	低	1-2 轮	2-5 分钟
中等图表	中	3-4 轮	5-10 分钟
复杂图表	高	5-8 轮	10-20 分钟

影响因素:

- 图表复杂度（元素数量、颜色种类）
- 模型响应速度
- 网络延迟
- 服务器性能

9. 常见问题排查

9.1 安装问题

问题 1: Python 版本不兼容

症状:

```
ERROR: This package requires Python 3.9 or higher
```

解决方案:

```
# 检查 Python 版本
python --version

# 安装正确版本的 Python
# 参考 3.1 节安装 Python 3.9+
```

问题 2: pip 安装依赖失败

症状:

```
ERROR: Could not find a version that satisfies the requirement
```

解决方案:

```
# 升级 pip
python -m pip install --upgrade pip

# 使用国内镜像
pip install -r backend/requirements.txt -i
https://pypi.tuna.tsinghua.edu.cn/simple

# 如果特定包安装失败，单独安装
pip install fastapi==0.104.1
```

问题 3: Node.js 依赖安装失败

症状:

```
npm ERR! code ECONNREFUSED
npm ERR! network request failed
```

解决方案:

```
# 清除 npm 缓存
npm cache clean --force

# 使用国内镜像
npm config set registry https://registry.npmmirror.com

# 重新安装
cd frontend
```



```
rm -rf node_modules package-lock.json
npm install
```

问题 4: Tesseract 找不到

症状:

```
TesseractNotFoundError: tesseract is not installed or it's not in your PATH
```

解决方案:

Windows:

```
# 1. 确认 Tesseract 已安装
# 2. 在 .env 中配置完整路径
TESSERACT_CMD=C:\Program Files\Tesseract-OCR\tesseract.exe

# 3. 或添加到系统 PATH
# 系统属性 → 环境变量 → Path → 添加 C:\Program Files\Tesseract-OCR
```

Linux/macOS:

```
# 安装 Tesseract
sudo apt install tesseract-ocr # Ubuntu/Debian
brew install tesseract         # macOS

# 查找路径
which tesseract

# 在 .env 中配置
TESSERACT_CMD=/usr/bin/tesseract
```

9.2 启动问题

问题 5: 端口被占用

症状:

```
ERROR: [Errno 48] Address already in use
```

解决方案:

查找占用端口的进程:

```
# Windows
netstat -ano | findstr :8001
taskkill /PID <PID> /F

# Linux/macOS
lsof -i :8001
kill -9 <PID>
```

或更改端口:

```
# .env
SERVER_PORT=8002
```

```
# frontend/.env
VITE_API_BASE_URL=http://localhost:8002/api
VITE_WS_URL=ws://localhost:8002/ws
```

问题 6: 后端启动失败

症状:

```
ModuleNotFoundError: No module named 'backend'
```

解决方案:

```
# 确保在项目根目录
pwd # 应显示 ../MultiAgentFrame-main

# 确保 Python 路径正确
export PYTHONPATH="${PYTHONPATH}:${pwd}" # Linux/macOS
set PYTHONPATH=%PYTHONPATH%;%CD%        # Windows

# 使用启动脚本
python scripts/start_service.py
```

问题 7: 前端无法连接后端

症状:

- 浏览器控制台显示 `net::ERR_CONNECTION_REFUSED`
- 或 `CORS policy` 错误

解决方案:

1. 确认后端已启动:

```
curl http://localhost:8001/health
```

2. 检查 CORS 配置: 编辑 `backend/main.py`, 确保前端地址在允许列表中:

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        "http://localhost:5173",
        "http://localhost:5174",
        "http://localhost:5175",
    ],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

3. 检查防火墙:

```
# Windows: 允许端口 8001
# 控制面板 → Windows Defender 防火墙 → 高级设置 → 入站规则

# Linux: 使用 ufw
sudo ufw allow 8001
```

9.3 运行时问题

问题 8: API Key 无效

症状:

```
Error: Invalid API key
Error: Authentication failed
```

解决方案:

1. 检查 `.env` 配置:

```
# 确认 API Key 正确无误
cat .env | grep API_KEY
```

```
# 确认没有多余的空格或引号
# 正确: QWEN_API_KEY=sk-abc123
# 错误: QWEN_API_KEY="sk-abc123"
# 错误: QWEN_API_KEY= sk-abc123
```

2. 验证 API Key:

```
# 通义千问
curl -X POST "https://dashscope.aliyuncs.com/api/v1/services/aigc/text-
generation/generation" \
  -H "Authorization: Bearer YOUR_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"model": "qwen-turbo", "input": {"messages": [{"role": "user", "content": "你
好"}]}}'
```

3. 检查配额:

- 登录对应平台查看 API 调用配额
- 确认账户余额充足

问题 9: 流水线执行失败

症状:

- 流水线卡在某个阶段
- 返回错误信息

解决方案:

1. 查看日志:

```
# 查看后端日志
tail -f backend/logs/app.log
tail -f backend/logs/error.log

# Windows
type backend\logs\error.log
```

2. 检查输入图片:

- 确保图片格式正确 (PNG/JPG)
- 确保图片大小不超过 10MB
- 确保图片内容清晰可识别

3. 调整参数:

```
# 降低阈值
threshold = 0.6 # 从 0.75 降低到 0.6

# 增加最大轮数
max_loops = 8 # 从 5 增加到 8
```

4. 切换模型:

```
# .env
# 尝试不同的模型提供商
MODEL_PROVIDER=openai # 或 gemini, doubao
```

问题 10: WebSocket 连接断开

症状:

- 实时进度不更新
- 控制台显示 WebSocket 错误

解决方案:

1. 检查 WebSocket 配置:

```
// frontend/.env
VITE_WS_URL=ws://localhost:8001/ws // 确保协议是 ws:// 不是 wss://
```

2. 检查代理配置: 如果使用 Nginx 代理, 确保 WebSocket 配置正确:

```
location /ws {
    proxy_pass http://localhost:8001;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "Upgrade";
}
```

3. 增加超时时间:

```
# backend/websocket/manager.py
# 增加心跳间隔
await asyncio.sleep(30) # 从 10 增加到 30
```

问题 11: 生成结果不理想

症状:

- 生成的图表与原图差异较大
- 验证分数很低

解决方案:

1. 优化输入图片:

- 使用高分辨率图片
- 确保图表清晰、无遮挡
- 避免复杂背景

2. 调整参数:

```
# 增加迭代轮数
max_loops = 10

# 降低阈值（允许更多轮次）
threshold = 0.65
```

3. 尝试不同模型: 不同模型在不同类型图表上表现不同，可以尝试切换：

```
MODEL_PROVIDER=qwen      # 适合中文图表
MODEL_PROVIDER=openai     # 适合复杂图表
MODEL_PROVIDER=gemini     # 适合多模态理解
```

4. 手动调整代码:

- 下载生成的代码
- 手动微调参数
- 重新运行验证

9.4 性能问题

问题 12: 执行速度慢

症状:

- 单次流水线执行超过 30 分钟
- 系统响应缓慢

解决方案:

1. 检查网络连接:

```
# 测试 API 延迟
```

```
ping dashscope.aliyuncs.com
```

2. 优化服务器配置:

```
# 增加 uvicorn workers
uvicorn backend.main:app --workers 4
```

3. 使用更快的模型:

```
# 使用更快的模型版本
MODEL_PROVIDER=qwen
# qwen-turbo 比 qwen-plus 更快
```

4. 减少迭代轮数:

```
max_loops = 3 # 从 5 减少到 3
```

问题 13: 内存占用过高

症状:

- 系统内存占用超过 4GB
- 出现 OOM (Out of Memory) 错误

解决方案:

1. 限制并发流水线:

```
# backend/services/pipeline_service.py
# 添加并发限制
MAX_CONCURRENT_PIPELINES = 2
```

2. 清理临时文件:

```
# 定期清理上传和输出目录
rm -rf storage/uploads/*
rm -rf storage/outputs/*
```

3. 优化图片处理:

```
# 压缩大图片
from PIL import Image

def resize_image(image_path, max_size=1920):
    img = Image.open(image_path)
    if max(img.size) > max_size:
        img.thumbnail((max_size, max_size))
        img.save(image_path)
```

9.5 日志分析

查看日志位置

```
# 应用日志
backend/logs/app.log

# 错误日志
backend/logs/error.log

# 实时查看
tail -f backend/logs/app.log
```

常见错误日志

1. 模型调用失败:

```
ERROR: Model API call failed: Connection timeout
```

→ 检查网络连接和 API Key

2. 文件处理错误:

```
ERROR: Failed to process image: Invalid image format
```

→ 检查图片格式和完整性

3. 验证器错误:

```
ERROR: Validator failed: Tesseract not found
```

→ 检查 Tesseract 安装和配置

10. 性能优化

10.1 后端优化

10.1.1 使用异步操作

```
# 使用 asyncio 并发处理
import asyncio

async def process_multiple_images(images):
    tasks = [process_image(img) for img in images]
    results = await asyncio.gather(*tasks)
    return results
```

10.1.2 添加缓存

```
from functools import lru_cache

@lru_cache(maxsize=100)
def get_image_features(image_path):
    # 缓存图片特征提取结果
    return extract_features(image_path)
```

10.1.3 数据库优化（如使用）

```
# 使用连接池
from sqlalchemy import create_engine
from sqlalchemy.pool import QueuePool

engine = create_engine(
    DATABASE_URL,
    poolclass=QueuePool,
    pool_size=10,
    max_overflow=20
)
```

10.2 前端优化

10.2.1 组件懒加载

```
// router/index.js
const routes = [
  {
    path: '/results',
```

```
    component: () => import('../views/ResultsView.vue') // 懒加载
  }
]
```

10.2.2 图片优化

```
// 图片懒加载
<img v-lazy="imageUrl" alt="Chart">

// 图片压缩
import imageCompression from 'browser-image-compression'

async function compressImage(file) {
  const options = {
    maxSizeMB: 1,
    maxWidthOrHeight: 1920
  }
  return await imageCompression(file, options)
}
```

10.2.3 防抖和节流

```
import { debounce } from 'lodash-es'

// 防抖：搜索输入
const handleSearch = debounce((query) => {
  searchAPI(query)
}, 300)

// 节流：滚动事件
const handleScroll = throttle(() => {
  updateScrollPosition()
}, 100)
```

10.3 系统级优化

10.3.1 使用 Redis 缓存

```
import redis

redis_client = redis.Redis(host='localhost', port=6379, db=0)

# 缓存流水线结果
def cache_pipeline_result(pipeline_id, result):
    redis_client.setex(
        f"pipeline:{pipeline_id}",
```

```
    3600, # 1小时过期
    json.dumps(result)
)
```

10.3.2 使用 CDN

```
# Nginx 配置静态资源缓存
location ~* \.(jpg|jpeg|png|gif|ico|css|js)$ {
    expires 1y;
    add_header Cache-Control "public, immutable";
}
```

10.3.3 负载均衡

```
upstream backend {
    server 127.0.0.1:8001;
    server 127.0.0.1:8002;
    server 127.0.0.1:8003;
}

server {
    location /api {
        proxy_pass http://backend;
    }
}
```

11. 安全建议

11.1 API Key 安全

11.1.1 环境变量管理

```
# 不要将 .env 文件提交到版本控制
echo ".env" >> .gitignore

# 使用环境变量管理工具
# 生产环境使用 Vault、AWS Secrets Manager 等
```

11.1.2 API Key 轮换

```
# 定期轮换 API Key
# 实现多 Key 轮询机制
```

```
API_KEYS = [
    os.getenv("QWEN_API_KEY_1"),
    os.getenv("QWEN_API_KEY_2"),
    os.getenv("QWEN_API_KEY_3"),
]

def get_next_api_key():
    # 轮询使用不同的 Key
    return API_KEYS[current_index % len(API_KEYS)]
```

11.2 文件上传安全

11.2.1 文件类型验证

```
from fastapi import UploadFile, HTTPException

ALLOWED_EXTENSIONS = {'.png', '.jpg', '.jpeg'}
MAX_FILE_SIZE = 10 * 1024 * 1024 # 10MB

async def validate_upload(file: UploadFile):
    # 检查文件扩展名
    ext = Path(file.filename).suffix.lower()
    if ext not in ALLOWED_EXTENSIONS:
        raise HTTPException(400, "不支持的文件类型")

    # 检查文件大小
    content = await file.read()
    if len(content) > MAX_FILE_SIZE:
        raise HTTPException(400, "文件过大")

    # 验证文件内容（魔数检查）
    if not content.startswith(b'\x89PNG') and not content.startswith(b'\xff\xd8'):
        raise HTTPException(400, "无效的图片文件")

    return content
```

11.2.2 文件名安全

```
import uuid
from pathlib import Path

def safe_filename(original_filename: str) -> str:
    # 使用 UUID 生成安全的文件名
    ext = Path(original_filename).suffix.lower()
    return f"{uuid.uuid4()} {ext}"
```

11.3 访问控制

11.3.1 添加认证

```
from fastapi import Depends, HTTPException, Header

async def verify_token(authorization: str = Header(None)):
    if not authorization or not authorization.startswith("Bearer "):
        raise HTTPException(401, "未授权")

    token = authorization.replace("Bearer ", "")
    # 验证 token
    if not is_valid_token(token):
        raise HTTPException(401, "无效的令牌")

    return token

@app.post("/api/upload")
async def upload(file: UploadFile, token: str = Depends(verify_token)):
    # 需要认证才能上传
    pass
```

11.3.2 速率限制

```
from slowapi import Limiter
from slowapi.util import get_remote_address

limiter = Limiter(key_func=get_remote_address)
app.state.limiter = limiter

@app.post("/api/pipeline/start")
@limiter.limit("10/minute") # 每分钟最多 10 次
async def start_pipeline(request: Request):
    pass
```

11.4 HTTPS 配置

11.4.1 使用 Let's Encrypt

```
# 安装 Certbot
sudo apt install certbot python3-certbot-nginx

# 获取证书
sudo certbot --nginx -d your-domain.com

# 自动续期
sudo certbot renew --dry-run
```

11.4.2 Nginx HTTPS 配置

```
server {  
    listen 443 ssl http2;  
    server_name your-domain.com;  
  
    ssl_certificate /etc/letsencrypt/live/your-domain.com/fullchain.pem;  
    ssl_certificate_key /etc/letsencrypt/live/your-domain.com/privkey.pem;  
  
    ssl_protocols TLSv1.2 TLSv1.3;  
    ssl_ciphers HIGH:!aNULL:!MD5;  
    ssl_prefer_server_ciphers on;  
  
    # 其他配置...  
}  
  
# HTTP 重定向到 HTTPS  
server {  
    listen 80;  
    server_name your-domain.com;  
    return 301 https://$server_name$request_uri;  
}
```

12. 维护与监控

12.1 日志管理

12.1.1 日志轮转

创建 `/etc/logrotate.d/multiagent`:

```
/path/to/MultiAgentFrame-main/backend/logs/*.log {  
    daily  
    rotate 7  
    compress  
    delaycompress  
    missingok  
    notifempty  
    create 0644 www-data www-data  
}
```

12.1.2 日志级别配置

```
# backend/utils/logger.py  
import logging
```

```
# 生产环境使用 INFO
logging.basicConfig(level=logging.INFO)

# 开发环境使用 DEBUG
# logging.basicConfig(level=logging.DEBUG)
```

12.2 监控指标

12.2.1 系统监控

```
# 安装监控工具
pip install prometheus-client

# 添加监控端点
from prometheus_client import Counter, Histogram, generate_latest

pipeline_counter = Counter('pipeline_total', 'Total pipelines executed')
pipeline_duration = Histogram('pipeline_duration_seconds', 'Pipeline execution time')

@app.get("/metrics")
async def metrics():
    return Response(generate_latest(), media_type="text/plain")
```

12.2.2 健康检查

```
@app.get("/health")
async def health_check():
    checks = {
        "status": "ok",
        "timestamp": datetime.now().isoformat(),
        "checks": {
            "database": check_database(),
            "storage": check_storage(),
            "api_key": check_api_key(),
        }
    }

    if all(checks["checks"].values()):
        return checks
    else:
        raise HTTPException(503, detail=checks)
```

12.3 备份策略

12.3.1 数据备份

```
#!/bin/bash
# backup.sh

BACKUP_DIR="/backups/multiagent"
DATE=$(date +%Y%m%d_%H%M%S)

# 备份上传文件
tar -czf "$BACKUP_DIR/uploads_$DATE.tar.gz" storage/uploads/

# 备份输出文件
tar -czf "$BACKUP_DIR/outputs_$DATE.tar.gz" storage/outputs/

# 备份配置文件
tar -czf "$BACKUP_DIR/config_$DATE.tar.gz" .env backend/config.py

# 删除 7 天前的备份
find "$BACKUP_DIR" -name "*.tar.gz" -mtime +7 -delete
```

12.3.2 定时备份

```
# 添加到 crontab
crontab -e

# 每天凌晨 2 点执行备份
0 2 * * * /path/to/backup.sh
```

12.4 更新维护

12.4.1 依赖更新

```
# 检查过期的包
pip list --outdated

# 更新特定包
pip install --upgrade fastapi

# 更新所有包（谨慎）
pip install --upgrade -r backend/requirements.txt
```

12.4.2 系统更新流程

1. 备份当前版本
2. 在测试环境验证
3. 更新依赖
4. 运行测试

5. 部署到生产环境

6. 监控运行状态

12.5 故障恢复

12.5.1 服务自动重启

```
# /etc/systemd/system/multiagent-backend.service
[Service]
Restart=always
RestartSec=10
StartLimitInterval=0
```

12.5.2 数据恢复

```
# 从备份恢复
tar -xzf /backups/multiagent/uploads_20240101_020000.tar.gz -C /

# 验证数据完整性
ls -lh storage/uploads/
```

附录

A. 快速参考命令

```
# 启动服务
python scripts/start_service.py

# 单独启动后端
cd backend && uvicorn main:app --reload --port 8001

# 单独启动前端
cd frontend && npm run dev

# 查看日志
tail -f backend/logs/app.log

# 健康检查
curl http://localhost:8001/health

# 清理临时文件
rm -rf storage/uploads/* storage/outputs/*
```

B. 环境变量完整列表

变量名	说明	默认值	必需
MODEL_PROVIDER	模型提供商	qwen	是
QWEN_API_KEY	通义千问 API Key	-	条件
OPENAI_API_KEY	OpenAI API Key	-	条件
GEMINI_API_KEY	Gemini API Key	-	条件
DOUBAO_API_KEY	豆包 API Key	-	条件
SERVER_HOST	服务器地址	localhost	否
SERVER_PORT	服务器端口	8001	否
TESSERACT_CMD	Tesseract 路径	-	是

C. API 端点列表

方法	端点	说明
GET	/health	健康检查
POST	/api/upload	上传图片
POST	/api/pipeline/start	启动流水线
GET	/api/pipeline/{id}/status	查询状态
GET	/api/results/{id}	获取结果
GET	/api/history	历史记录
GET	/api/download/{id}	下载文件
WS	/ws/{id}	WebSocket 连接